LÝ THUYẾT ĐỒ THỊ CƠ BẢN

Nhóm 6

Nguyễn Văn Phú **Hà Minh Đức** **Nguyễn Tấn Đạt**

THPT Chuyên ĐHV

Ngày 18 tháng 3 năm 2026

Mục lục

1. Giới thiệu về đồ thị
2. Khái niệm về đồ thị
3. Phân loại đồ thị
4. Cách dựng đồ thị
5. Thuật toán DFS
6. Thuật toán BFS
7. Tổng kết

1. Giới thiệu về đồ thị

Giới thiệu

Trước khi đến với **lý thuyết đồ thị**, ta xét một bài toán nổi tiếng:

Giới thiệu

Trước khi đến với **lý thuyết đồ thị**, ta xét một bài toán nổi tiếng:

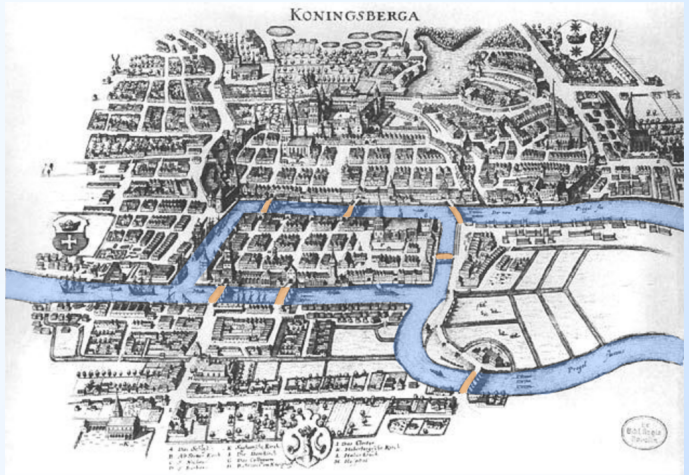
→ Thành phố **Königsberg** (nay là Kaliningrad, Nga) nằm hai bên sông Pregel, gồm 2 hòn đảo lớn.

Giới thiệu

Trước khi đến với **lý thuyết đồ thị**, ta xét một bài toán nổi tiếng:

→ Thành phố **Königsberg** (nay là Kaliningrad, Nga) nằm hai bên sông Pregel, gồm 2 hòn đảo lớn.

→ Các khu vực này được nối với nhau bởi **7 cây cầu**.

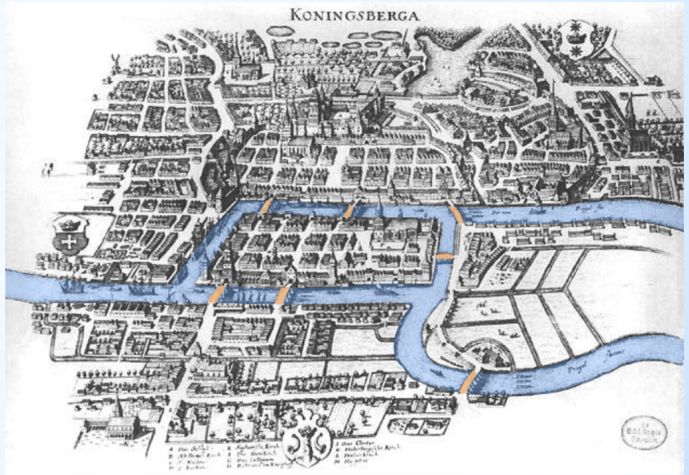


Giới thiệu

Trước khi đến với **lý thuyết đồ thị**, ta xét một bài toán nổi tiếng:

→ Thành phố **Königsberg** (nay là Kaliningrad, Nga) nằm hai bên sông Pregel, gồm 2 hòn đảo lớn.

→ Các khu vực này được nối với nhau bởi **7 cây cầu**.



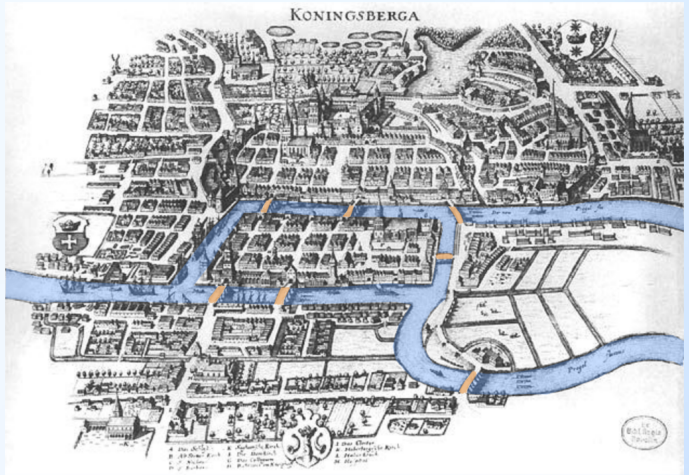
→ Câu hỏi đặt ra: Có tồn tại đường đi qua mỗi cây cầu **đúng một lần** hay không?

Giới thiệu

Trước khi đến với **lý thuyết đồ thị**, ta xét một bài toán nổi tiếng:

→ Thành phố **Königsberg** (nay là Kaliningrad, Nga) nằm hai bên sông Pregel, gồm 2 hòn đảo lớn.

→ Các khu vực này được nối với nhau bởi **7 cây cầu**.



→ Câu hỏi đặt ra: Có tồn tại đường đi qua mỗi cây cầu **đúng một lần** hay không?

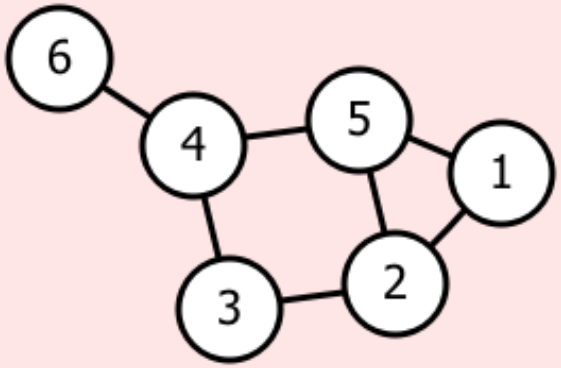
→ Bài toán này đã được **Leonhard Euler** giải vào thế kỉ XVIII.

→ Từ đó, **lý thuyết đồ thị** ra đời.

2. Khái niệm về đồ thị

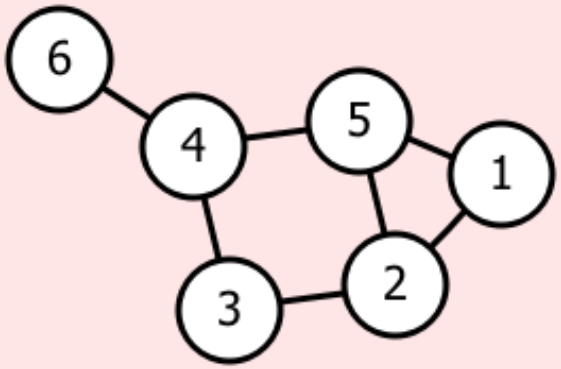
Định Nghĩa

Hình ảnh dưới đây là một ví dụ về **đồ thị**:



Định Nghĩa

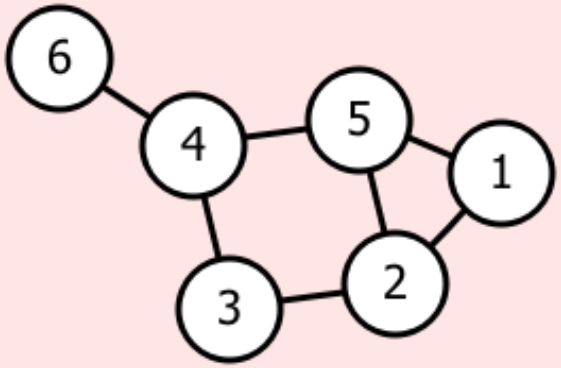
Hình ảnh dưới đây là một ví dụ về **đồ thị**:



- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.

Định Nghĩa

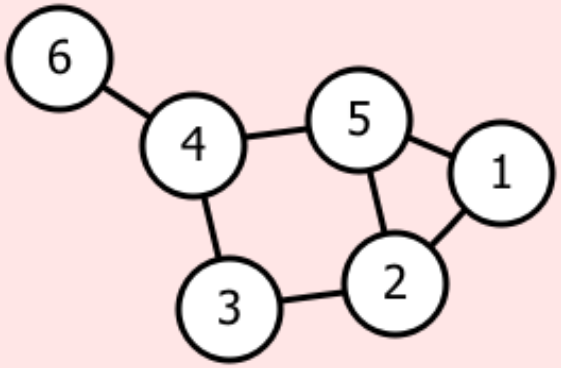
Hình ảnh dưới đây là một ví dụ về **đồ thị**:



- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.
- Các đoạn nối giữa các đỉnh gọi là **cạnh (edges)**.

Định Nghĩa

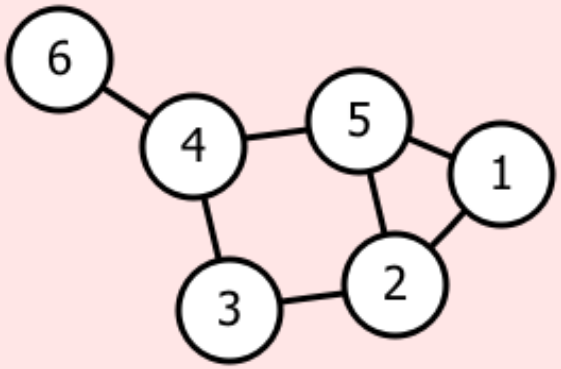
Hình ảnh dưới đây là một ví dụ về **đồ thị**:



- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.
- Các đoạn nối giữa các đỉnh gọi là **cạnh (edges)**.
- Đồ thị là tập hợp các đỉnh được nối với nhau bởi các cạnh.

Định Nghĩa

Hình ảnh dưới đây là một ví dụ về **đồ thị**:

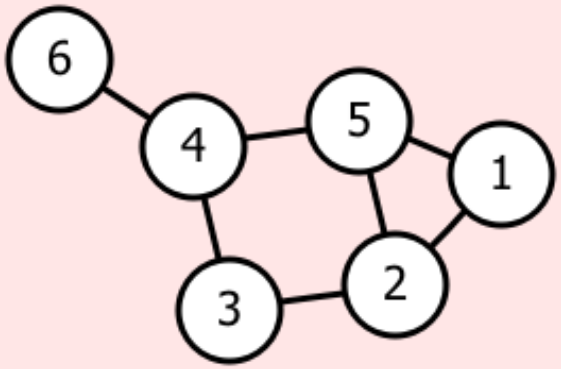


- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.
- Các đoạn nối giữa các đỉnh gọi là **cạnh (edges)**.
- Đồ thị là tập hợp các đỉnh được nối với nhau bởi các cạnh.
- Ký hiệu là:

$$G = (V, E)$$

Định Nghĩa

Hình ảnh dưới đây là một ví dụ về **đồ thị**:



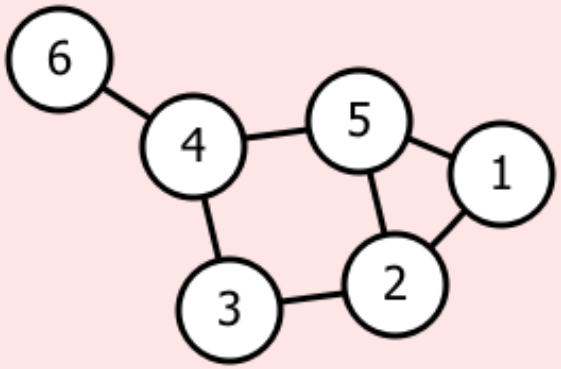
- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.
- Các đoạn nối giữa các đỉnh gọi là **cạnh (edges)**.
- Đồ thị là tập hợp các đỉnh được nối với nhau bởi các cạnh.
- Ký hiệu là:

$$G = (V, E)$$

- V: tập các đỉnh, E: tập các cạnh.
- Mỗi cạnh có dạng $\{u, v\}$.

Định Nghĩa

Hình ảnh dưới đây là một ví dụ về **đồ thị**:



- Các vòng tròn gọi là **đỉnh (vertices/nodes)**.
- Các đoạn nối giữa các đỉnh gọi là **cạnh (edges)**.
- Đồ thị là tập hợp các đỉnh được nối với nhau bởi các cạnh.
- Ký hiệu là:

$$G = (V, E)$$

- V: tập các đỉnh, E: tập các cạnh.
- Mỗi cạnh có dạng $\{u, v\}$.
- Ví dụ: $G = (\{1, 2, 3\}, \{\{1, 2\}, \{2, 3\}\})$

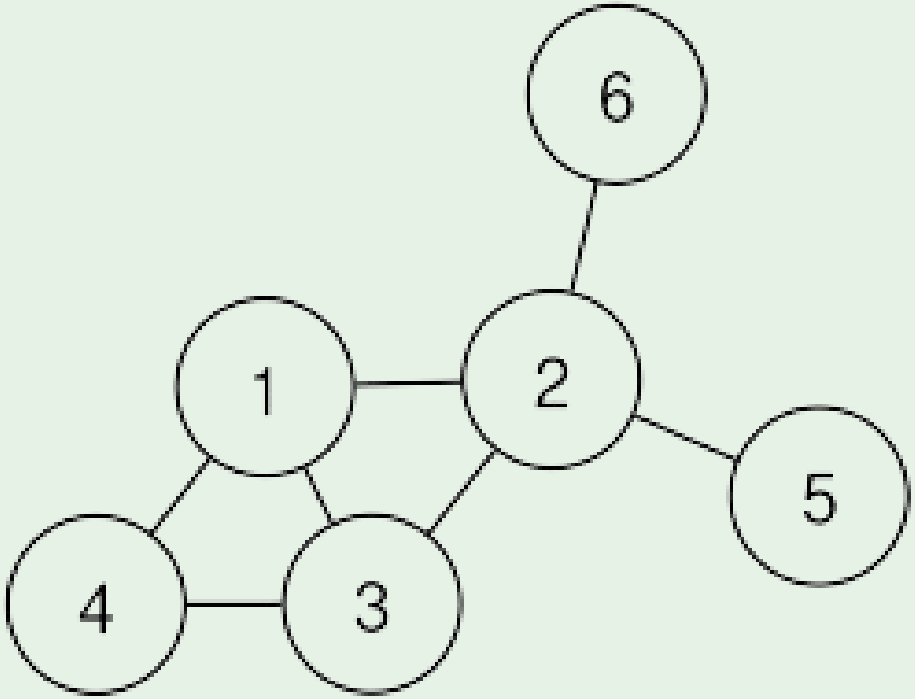
3. Phân loại đồ thị

CÁC DẠNG ĐỒ THỊ

Đơn đồ thị

- → Không có khuyên.
- → Không có cạnh song song.
- → Không trọng số.

Ví dụ về đơn đồ thị



CÁC DẠNG ĐỒ THỊ

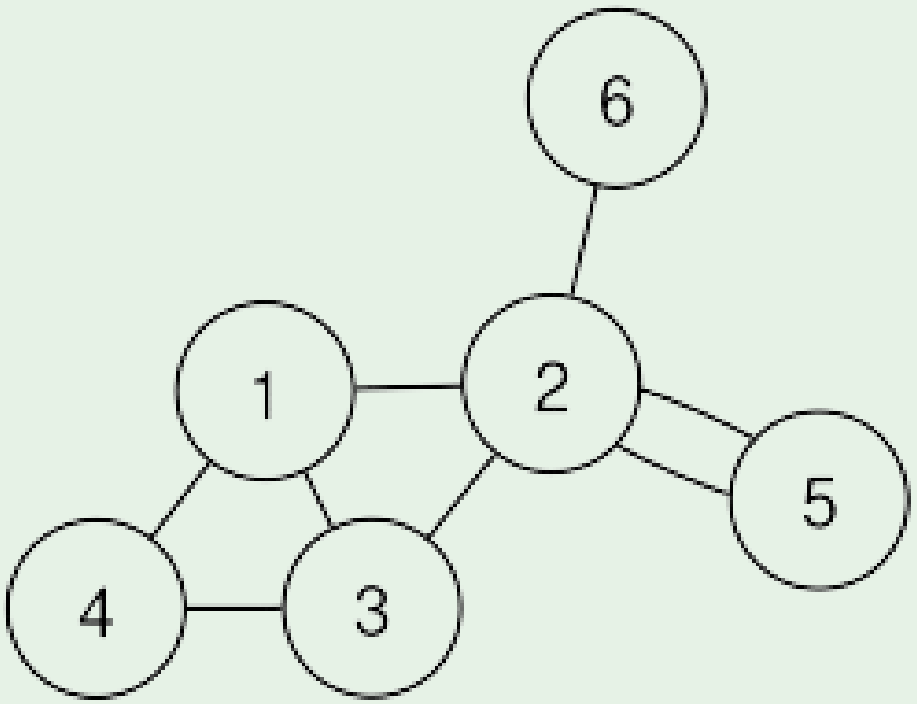
Đa đồ thị:

- Có cạnh song song.
- Có thể có khuyên.

Ghi chú:

- Khuyên: (u, u)
- Song song: nhiều cạnh giữa 2 đỉnh

Ví dụ về đa đồ thị

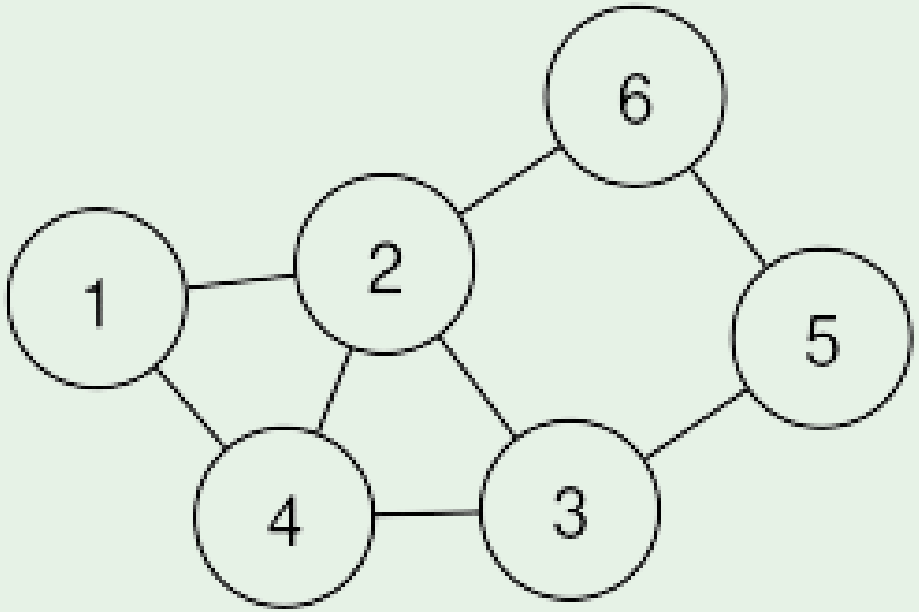


CÁC DẠNG ĐỒ THỊ

Đồ thị vô hướng

- Cạnh không có hướng.
- $uv = vu$.

Ví dụ về đồ thị vô hướng

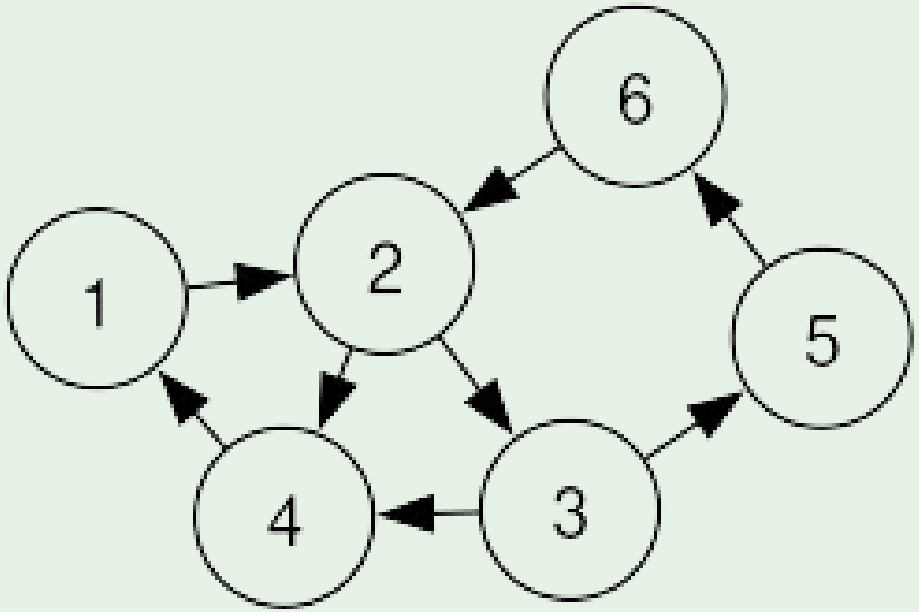


CÁC DẠNG ĐỒ THỊ

Đồ thị có hướng:

- Cạnh có hướng.
- $uv \neq vu$.

Ví dụ về đồ thị có hướng



CÁC DẠNG ĐỒ THỊ

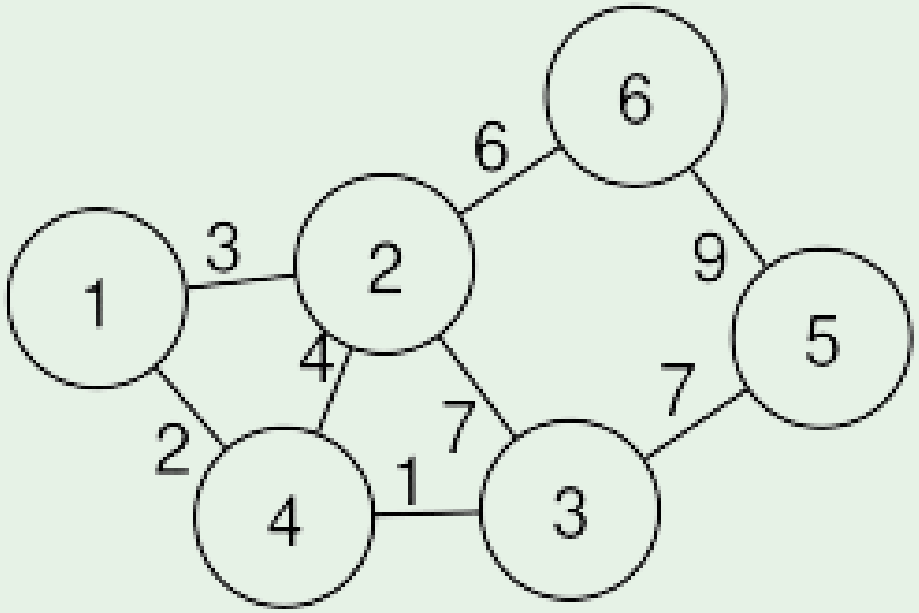
Đồ thị có trọng số:

- Mỗi cạnh có giá trị.

Đồ thị không trọng số:

- Cạnh không có giá trị.

Ví dụ về đồ thị có trọng số



4. Cách dựng đồ thị

MA TRẬN KỀ (Adjacency Matrix)

Khái niệm:

- Ma trận kề dùng để biểu diễn đồ thị
- Với đồ thị $G(V, E)$, ta tạo ma trận A

MA TRẬN KỀ (Adjacency Matrix)

Khái niệm:

- Ma trận kề dùng để biểu diễn đồ thị
- Với đồ thị $G(V, E)$, ta tạo ma trận A

Ý tưởng:

- $A[u][v] = 1$ nếu có cạnh
- $A[u][v] = 0$ nếu không có cạnh

MA TRẬN KỀ (Adjacency Matrix)

Khái niệm:

- Ma trận kề dùng để biểu diễn đồ thị
- Với đồ thị $G(V, E)$, ta tạo ma trận A

Ý tưởng:

- $A[u][v] = 1$ nếu có cạnh
- $A[u][v] = 0$ nếu không có cạnh

Lưu ý:

- Có trọng số $\rightarrow$ lưu w
- Vô hướng $\rightarrow A[u][v] = A[v][u]$

MA TRẬN KỀ (Adjacency Matrix)

Khái niệm:

- Ma trận kề dùng để biểu diễn đồ thị
- Với đồ thị $G(V, E)$, ta tạo ma trận A

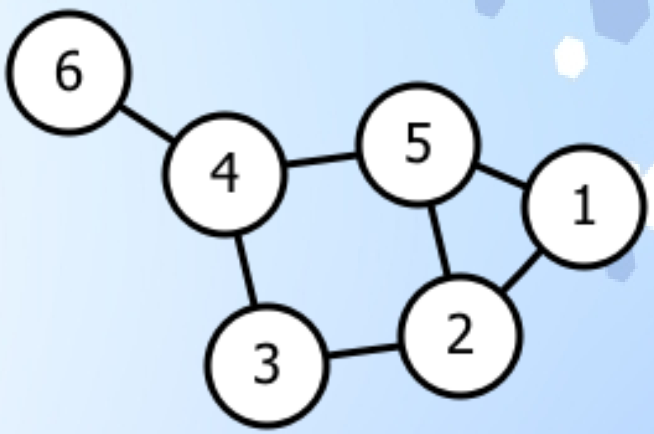
Ý tưởng:

- $A[u][v] = 1$ nếu có cạnh
- $A[u][v] = 0$ nếu không có cạnh

Lưu ý:

- Có trọng số $\rightarrow$ lưu w
- Vô hướng $\rightarrow A[u][v] = A[v][u]$

Ví dụ đồ thị:



MA TRẬN KỀ (Adjacency Matrix)

Khái niệm:

- Ma trận kề dùng để biểu diễn đồ thị
- Với đồ thị $G(V, E)$, ta tạo ma trận A

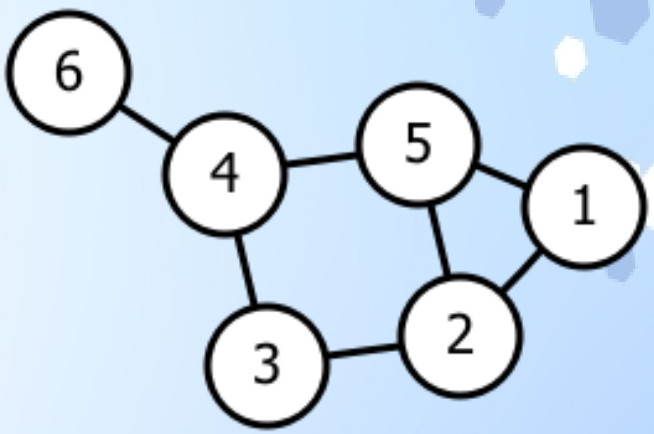
Ý tưởng:

- $A[u][v] = 1$ nếu có cạnh
- $A[u][v] = 0$ nếu không có cạnh

Lưu ý:

- Có trọng số $\rightarrow$ lưu w
- Vô hướng $\rightarrow A[u][v] = A[v][u]$

Ví dụ đồ thị:



Ví dụ dựng ma trận kề:

	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	1	0	1	0
3	0	1	0	1	0	0
4	0	0	1	0	1	1
5	1	1	0	1	0	0
6	0	0	0	1	0	0

Code minh họa - Ma trận kề

C++

```cpp
int adj[MAXN][MAXN] = {};

for (int i = 1; i <= m; i++)
{
    int u, v;
    cin >> u >> v;
    adj[u][v] = adj[v][u] = 1;
}
```

Python

```python
adj = [[0]*(n+1) for _ in range(n+1)]

for _ in range(m):
    u, v = map(int, input().split())
    adj[u][v] = adj[v][u] = 1
```

DANH SÁCH KỀ (Adjacency List)

Khái niệm:

- Danh sách kề dùng để biểu diễn đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề với nó

DANH SÁCH KỀ (Adjacency List)

Khái niệm:

- Danh sách kề dùng để biểu diễn đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề với nó

Ý tưởng:

- Với mỗi đỉnh u, lưu các đỉnh v kề với u
- Thường dùng vector / list trong code

DANH SÁCH KỀ (Adjacency List)

Khái niệm:

- Danh sách kề dùng để biểu diễn đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề với nó

Ý tưởng:

- Với mỗi đỉnh u, lưu các đỉnh v kề với u
- Thường dùng vector / list trong code

Lưu ý:

- Có trọng số $\rightarrow$ lưu cặp (v, w)

DANH SÁCH KỀ (Adjacency List)

Khái niệm:

- Danh sách kề dùng để biểu diễn đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề với nó

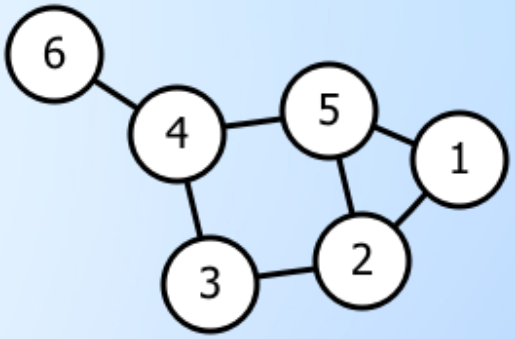
Ý tưởng:

- Với mỗi đỉnh u, lưu các đỉnh v kề với u
- Thường dùng vector / list trong code

Lưu ý:

- Có trọng số $\rightarrow$ lưu cặp (v, w)

Ví dụ đồ thị:



DANH SÁCH KỀ (Adjacency List)

Khái niệm:

- Danh sách kề dùng để biểu diễn đồ thị
- Mỗi đỉnh lưu danh sách các đỉnh kề với nó

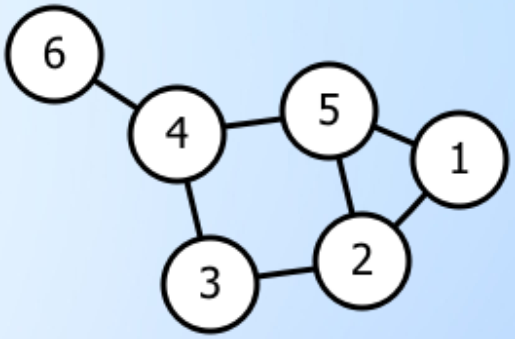
Ý tưởng:

- Với mỗi đỉnh u, lưu các đỉnh v kề với u
- Thường dùng vector / list trong code

Lưu ý:

- Có trọng số $\rightarrow$ lưu cặp (v, w)

Ví dụ đồ thị:



Danh sách kề tương ứng:

- $1 : 2, 5$
- $2 : 1, 3, 5$
- $3 : 2, 4$
- $4 : 3, 5, 6$
- $5 : 1, 2, 4$
- $6 : 4$

Code minh họa - Danh sách kề

C++

```cpp
vector<int> adj[MAXN];

for (int i = 1; i <= m; i++)
{
    int u, v;
    cin >> u >> v;
    adj[u].push_back(v);
    adj[v].push_back(u);
}
```

Python

```python
adj = [[] for _ in range(n+1)]

for _ in range(m):
    u, v = map(int, input().split())
    adj[u].append(v)
    adj[v].append(u)
```

DANH SÁCH CẠNH (Edge List)

Khái niệm:

- Danh sách cạnh lưu trực tiếp các cạnh của đồ thị
- Mỗi cạnh là một cặp đỉnh (u, v)

DANH SÁCH CẠNH (Edge List)

Khái niệm:

- Danh sách cạnh lưu trực tiếp các cạnh của đồ thị
- Mỗi cạnh là một cặp đỉnh (u, v)

Ý tưởng:

- Lưu danh sách các cạnh trong mảng / vector
- Mỗi phần tử là một cặp (u, v)

DANH SÁCH CẠNH (Edge List)

Khái niệm:

- Danh sách cạnh lưu trực tiếp các cạnh của đồ thị
- Mỗi cạnh là một cặp đỉnh (u, v)

Ý tưởng:

- Lưu danh sách các cạnh trong mảng / vector
- Mỗi phần tử là một cặp (u, v)

Lưu ý:

- Có trọng số $\rightarrow$ lưu (u, v, w)
- Vô hướng $\rightarrow$ mỗi cạnh chỉ cần lưu một lần

DANH SÁCH CẠNH (Edge List)

Khái niệm:

- Danh sách cạnh lưu trực tiếp các cạnh của đồ thị
- Mỗi cạnh là một cặp đỉnh (u, v)

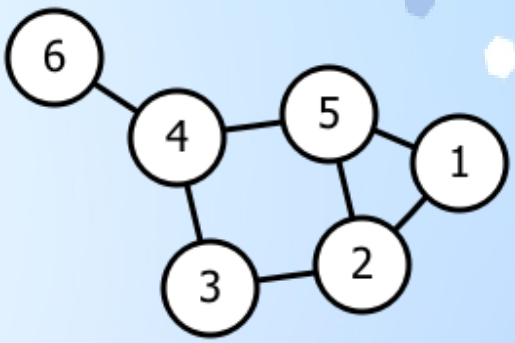
Ý tưởng:

- Lưu danh sách các cạnh trong mảng / vector
- Mỗi phần tử là một cặp (u, v)

Lưu ý:

- Có trọng số $\rightarrow$ lưu (u, v, w)
- Vô hướng $\rightarrow$ mỗi cạnh chỉ cần lưu một lần

Ví dụ đồ thị:



DANH SÁCH CẠNH (Edge List)

Khái niệm:

- Danh sách cạnh lưu trực tiếp các cạnh của đồ thị
- Mỗi cạnh là một cặp đỉnh (u, v)

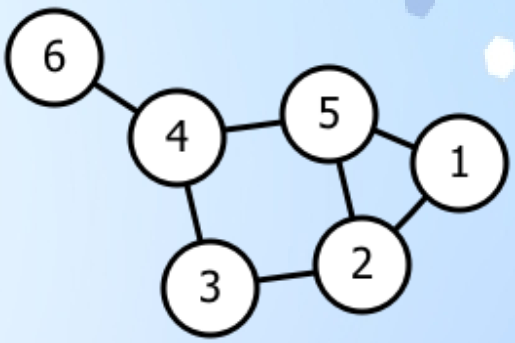
Ý tưởng:

- Lưu danh sách các cạnh trong mảng / vector
- Mỗi phần tử là một cặp (u, v)

Lưu ý:

- Có trọng số $\rightarrow$ lưu (u, v, w)
- Vô hướng $\rightarrow$ mỗi cạnh chỉ cần lưu một lần

Ví dụ đồ thị:



Danh sách cạnh tương ứng:

- $(1, 2)$
- $(1, 5)$
- $(2, 3)$
- $(2, 5)$
- $(3, 4)$
- $(4, 5)$
- $(4, 6)$

Code minh họa - Danh sách cạnh

C++

```cpp
int u[MAXM], v[MAXM];

for (int i = 1; i <= m; i++) {
    cin >> u[i] >> v[i];
}
```

Python

```python
u = [0]*(m+1)
v = [0]*(m+1)

for i in range(1, m+1):
    u[i], v[i] = map(int, input().split())
```

5. Thuật toán DFS

MỞ ĐẦU

Bài viết này sẽ giúp bạn tìm hiểu về cây **DFS** (Depth First Search Tree – DFS Tree). Cây **DFS** là một ứng dụng quan trọng có được từ kỹ thuật duyệt đồ thị ưu tiên chiều sâu, giúp giải các bài toán tìm khớp cầu và thành phần liên thông mạnh.

MỞ ĐẦU

Bài viết này sẽ giúp bạn tìm hiểu về cây **DFS** (Depth First Search Tree – DFS Tree). Cây **DFS** là một ứng dụng quan trọng có được từ kỹ thuật duyệt đồ thị ưu tiên chiều sâu, giúp giải các bài toán tìm khớp cầu và thành phần liên thông mạnh.

Ứng dụng chuyên sâu:

- Phát hiện **chu trình** (Cycle detection)
- Phân tích các thành phần **liên thông** (Connected)
- Sắp xếp **Topo** (Topological Sort).
- Giải nhiều bài toán **DP trên cây** kinh điển
- Cũng như nhiều ứng dụng khác nữa ...

Ý tưởng và cấu trúc

- Sử dụng **ngăn xếp (Stack)** hoặc **Đệ quy**
- Thăm một đỉnh kề, sau đó tiếp tục thăm ngay đỉnh kề của đỉnh đó cho đến khi gặp ngõ cụt mới **quai lui (backtrack).**

Ý tưởng và cấu trúc

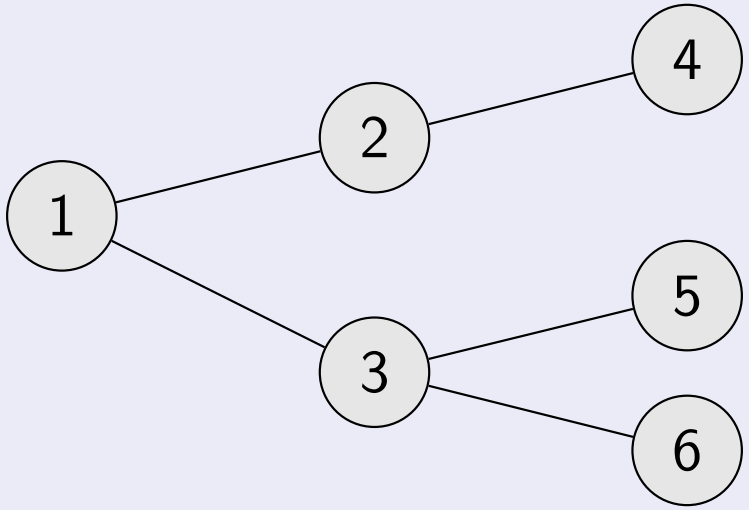
- Sử dụng **ngăn xếp (Stack)** hoặc **Đệ quy**
- Thăm một đỉnh kề, sau đó tiếp tục thăm ngay đỉnh kề của đỉnh đó cho đến khi gặp ngõ cụt mới **quai lui (backtrack).**

Cách hoạt động

- Bắt đầu từ một đỉnh
- Đi sang một đỉnh kề **chưa thăm**
- Lặp lại cho đến khi **không đi tiếp được**
- Quay lui lại đỉnh trước
- Tiếp tục với nhánh khác cho đến khi duyệt hết toàn bộ đỉnh của đồ thị

MINH HỌA

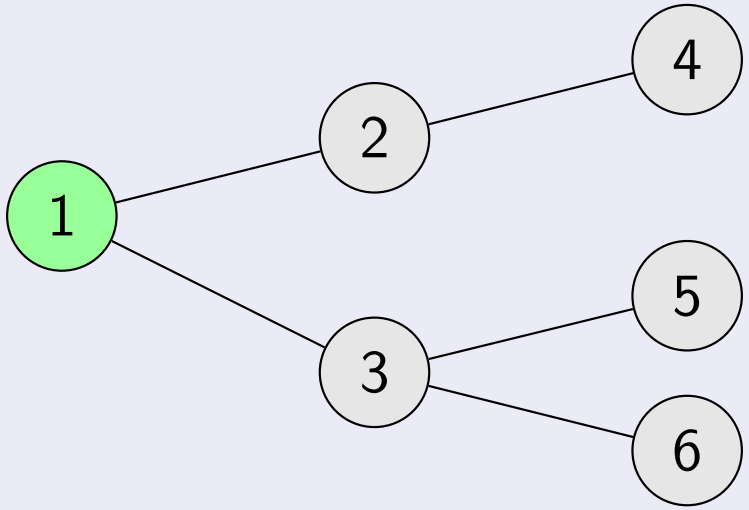
Minh họa cách hoạt động của 1 cây **DFS**



Ở đây là 1 đồ thị rỗng chưa có đỉnh nào được thăm.

MINH HỌA

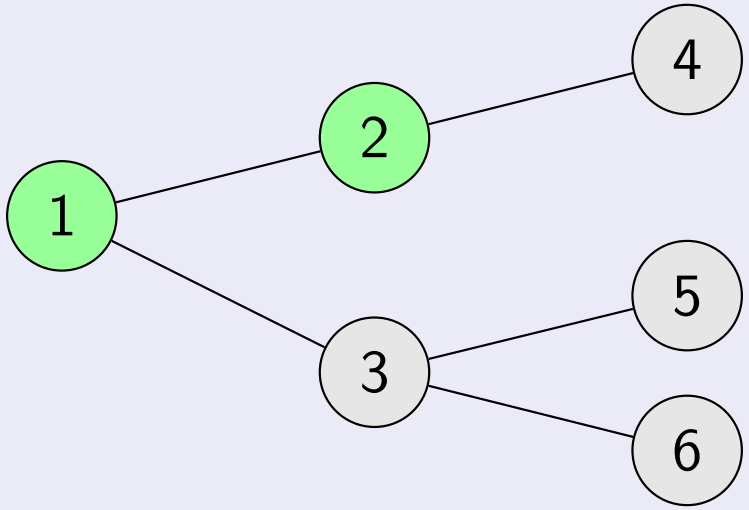
Minh họa cách hoạt động của 1 cây **DFS**



Bước đầu tiên sẽ thăm đỉnh đầu tiên được gọi, trong trường hợp này ta giả sử là 1

MINH HỌA

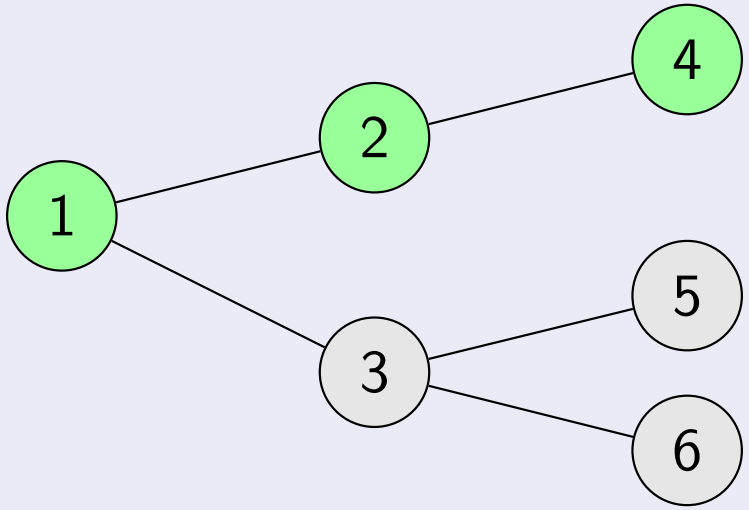
Minh họa cách hoạt động của 1 cây **DFS**



Từ đỉnh 1 thì DFS tiếp tục chạy đến nhánh 2

MINH HỌA

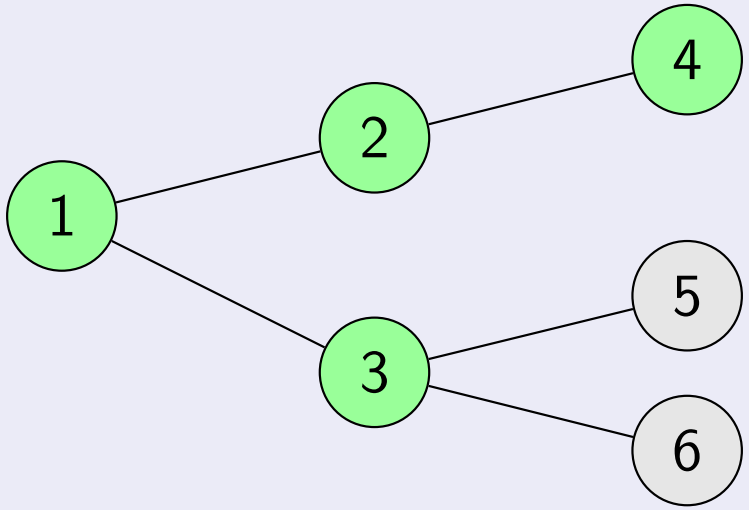
Minh họa cách hoạt động của 1 cây **DFS**



Tiếp tục duyệt đến đỉnh 4

MINH HỌA

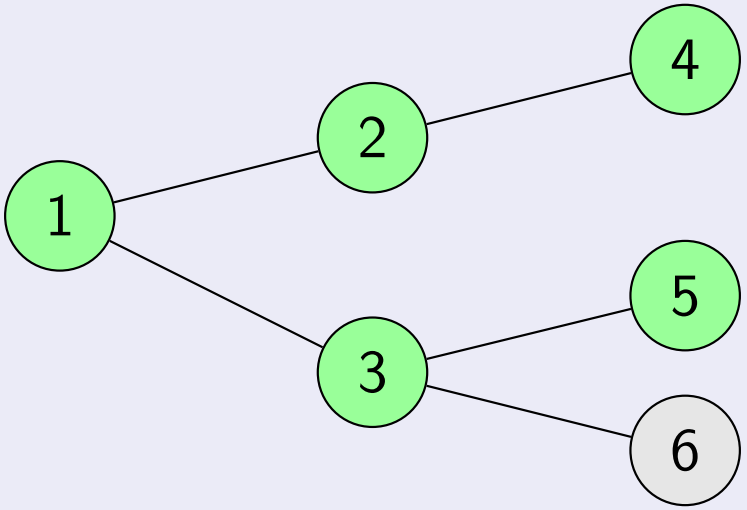
Minh họa cách hoạt động của 1 cây **DFS**



khi duyệt xong đỉnh 4 và không còn đỉnh nào ở nhánh để thăm nó sẽ quay lui lại đi tiếp đến cách nhánh khác để tiếp tục duyệt

MINH HỌA

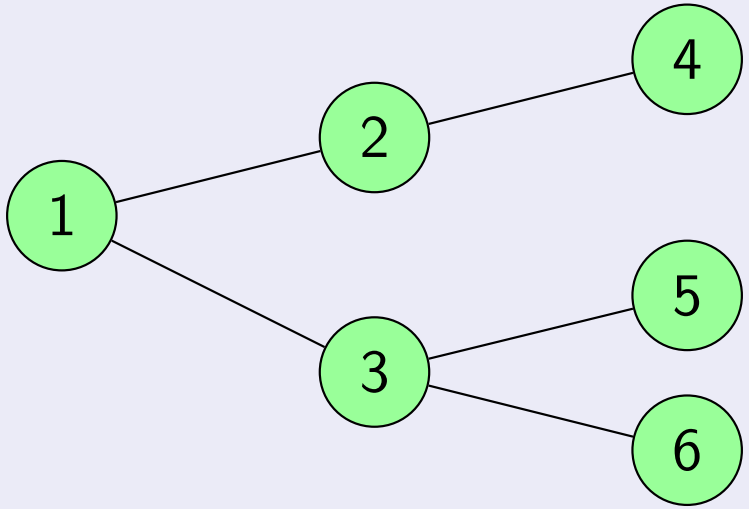
Minh họa cách hoạt động của 1 cây **DFS**



Tiếp tục sẽ theo nhánh của 3 đi đến 5

MINH HỌA

Minh họa cách hoạt động của 1 cây **DFS**



Từ 5 không thể đi tiếp theo nhánh đó, dfs sẽ quay trở về 3 và đi đến thăm 6 từ đó và kết thúc.

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ Ý nghĩa của cái tên **DFS** (duyệt theo chiều sâu) tức là nó sẽ đi sâu nhất có thể từ 1 nhánh gốc của đỉnh được gọi trước, sau đó mới duyệt đến các nhánh khác.

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ Ý nghĩa của cái tên **DFS** (duyệt theo chiều sâu) tức là nó sẽ đi sâu nhất có thể từ 1 nhánh gốc của đỉnh được gọi trước, sau đó mới duyệt đến các nhánh khác.

→ Nếu sử dụng **DFS** cho 1 đồ thị liên thông thì toàn bộ cách đỉnh của đồ thị đều sẽ được thăm ít nhất 1 lần.

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ Ý nghĩa của cái tên **DFS** (duyệt theo chiều sâu) tức là nó sẽ đi sâu nhất có thể từ 1 nhánh gốc của đỉnh được gọi trước, sau đó mới duyệt đến các nhánh khác.

→ Nếu sử dụng **DFS** cho 1 đồ thị liên thông thì toàn bộ cách đỉnh của đồ thị đều sẽ được thăm ít nhất 1 lần.

→ Theo mã giả cho cách hoạt động trên thì hàm DFS hoạt động với đệ quy cùng với một mảng bất kì (thông thường là **vis**, **visited**, **hoặc là bất kì cái tên nào khác mà bạn có thể nghĩ ra**) dùng để biểu diễn rằng đỉnh n đã được thăm thì độ phức tạp của thuật toán DFS xấp xỉ **O(V + E)**

Mã giả

```cpp
const ll N = 1e6 + 5;
bool vis[N] = {};
vector<ll> adj[N];
void dfs(ll u) {
    vis[u] = 1;
    //do something
    for (ll v : adj[u]) {
        //do something 1
        if (!vis[v]) dfs(v);
        //do something 2
    }
}
```

c++

```python
N = 10**6 + 5
vis = [False] * N
adj = [[] for _ in range(N)]

def dfs(u):
    vis[u] = True
    # do something
    for v in adj[u]:
        # do something 1
        if not vis[v]:
            dfs(v)
        # do something 2
```

python

- Trên đây là mã giả của DFS trong cả 2 phiên bản c++ và python (2 ngôn ngữ phổ biến hiện nay trong lĩnh vực lập trình thi đấu).
- Phần **do something** bạn có thể thêm những câu lệnh khác tùy theo mục đích sử dụng
- Vậy DFS là 1 hàm đệ quy giúp duyệt toàn bộ đồ thị theo chiều sâu

6. Thuật toán BFS

BFS - Duyệt theo chiều rộng

MỞ ĐẦU

Bài viết này sẽ giúp bạn tìm hiểu về cây **BFS** (Breadth First Search Tree – BFS Tree). Cây **BFS** là một ứng dụng quan trọng có được từ kỹ thuật duyệt đồ thị theo chiều rộng, giúp giải các bài toán tìm đường đi ngắn nhất trên đồ thị không trọng số và phân tích thành phần liên thông.

BFS - Duyệt theo chiều rộng

MỞ ĐẦU

Bài viết này sẽ giúp bạn tìm hiểu về cây **BFS** (Breadth First Search Tree – BFS Tree). Cây **BFS** là một ứng dụng quan trọng có được từ kỹ thuật duyệt đồ thị theo chiều rộng, giúp giải các bài toán tìm đường đi ngắn nhất trên đồ thị không trọng số và phân tích thành phần liên thông.

Ứng dụng chuyên sâu:

- Tìm **đường đi ngắn nhất** (Shortest Path - unweighted)
- Phân tích các thành phần **liên thông** (Connected)
- Kiểm tra đồ thị **hai phía** (Bipartite)
- Lan truyền theo **từng lớp** (Level Order Traversal)
- Cũng như nhiều ứng dụng khác nữa ...

Ý tưởng và cấu trúc

- Sử dụng **hàng đợi (Queue)**
- Thăm tất cả các đỉnh kề của một đỉnh trước khi đi tiếp sang các đỉnh ở mức sâu hơn

Ý tưởng và cấu trúc

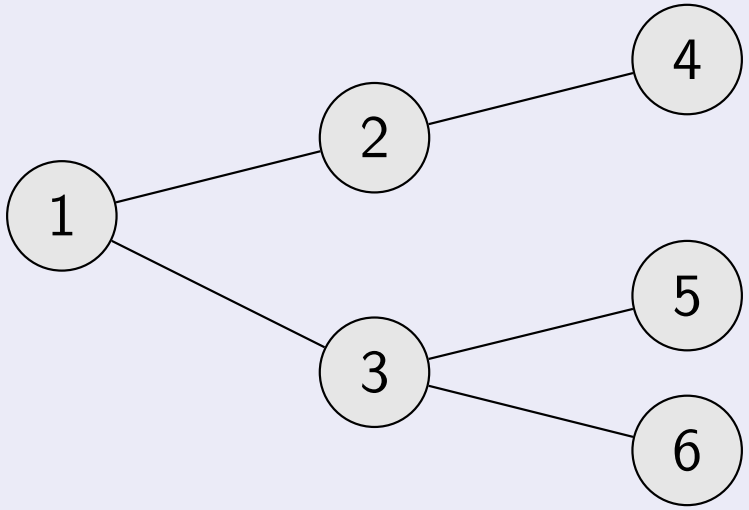
- Sử dụng **hàng đợi (Queue)**
- Thăm tất cả các đỉnh kề của một đỉnh trước khi đi tiếp sang các đỉnh ở mức sâu hơn

Cách hoạt động

- Bắt đầu từ một đỉnh
- Đưa đỉnh đó vào **queue**
- Lấy đỉnh ra và thăm tất cả các đỉnh kề **chưa thăm**
- Đưa các đỉnh đó vào queue
- Lặp lại cho đến khi queue rỗng

MINH HỌA

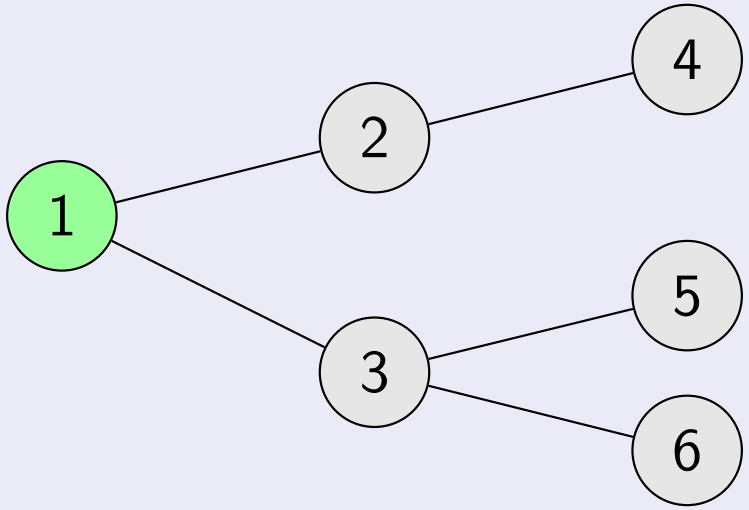
Minh họa cách hoạt động của 1 cây **BFS**



Ở đây là 1 đồ thị rỗng chưa có đỉnh nào được thăm.

MINH HỌA

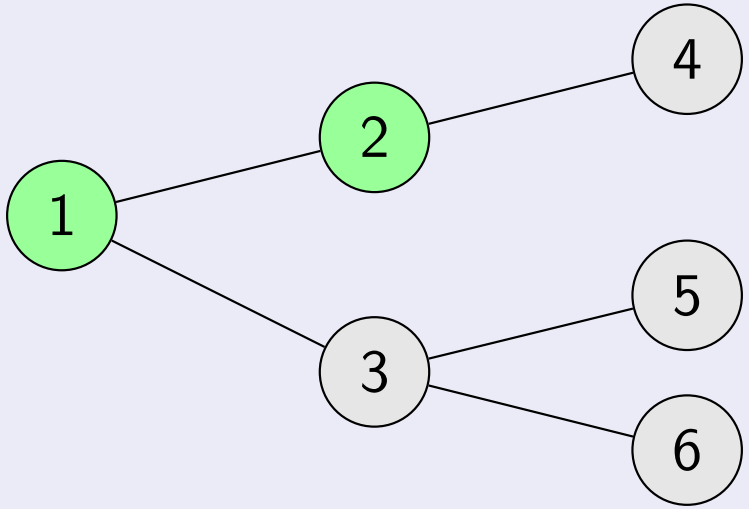
Minh họa cách hoạt động của 1 cây **BFS**



Bắt đầu từ đỉnh 1 và đưa vào queue

MINH HỌA

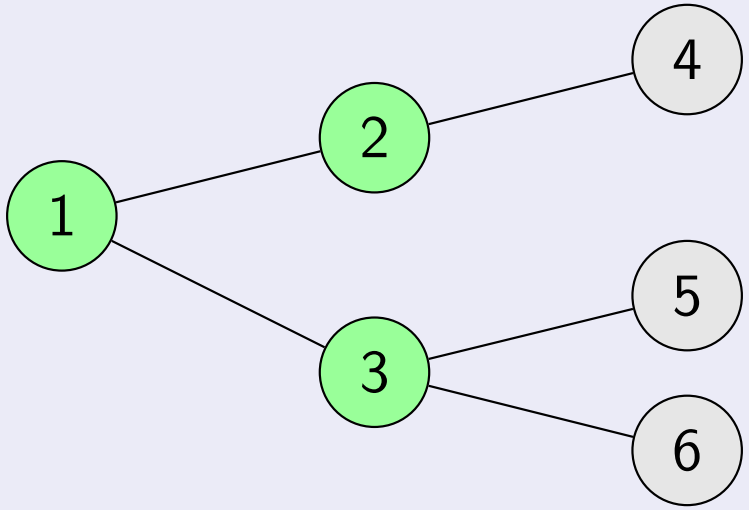
Minh họa cách hoạt động của 1 cây **BFS**



Thăm tất cả các đỉnh kề của 1 bắt đầu từ 2

MINH HỌA

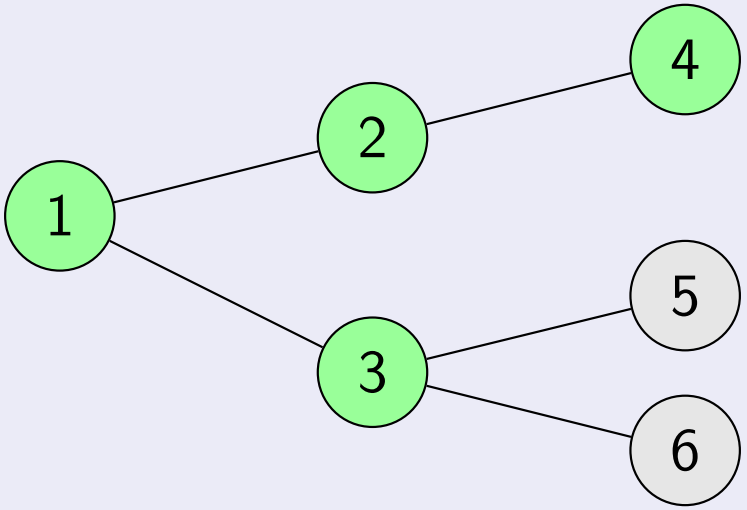
Minh họa cách hoạt động của 1 cây **BFS**



Tiếp tục thăm đỉnh kề của 1 là 3

MINH HỌA

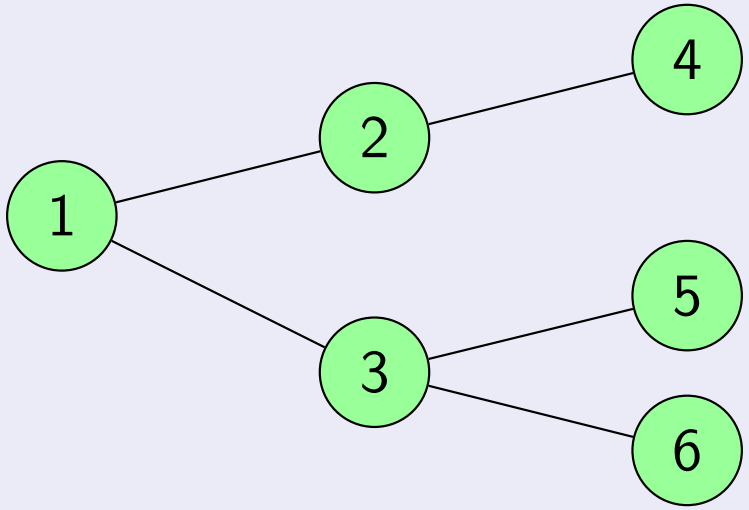
Minh họa cách hoạt động của 1 cây **BFS**



Tiếp theo xử lý đỉnh 2 và thăm đỉnh 4

MINH HỌA

Minh họa cách hoạt động của 1 cây **BFS**



Tiếp tục xử lý đỉnh 3 và thăm các đỉnh 5, 6 $\rightarrow$ kết thúc

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ BFS duyệt theo từng **lớp (level)**, tức là thăm hết các đỉnh gần trước rồi mới đi xa hơn.

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ BFS duyệt theo từng **lớp (level)**, tức là thăm hết các đỉnh gần trước rồi mới đi xa hơn.

→ Nếu sử dụng **BFS** cho 1 đồ thị liên thông thì toàn bộ các đỉnh đều sẽ được thăm.

Tổng quát

Từ minh họa trên ta có thể nhận thấy rằng:

→ BFS duyệt theo từng **lớp (level)**, tức là thăm hết các đỉnh gần trước rồi mới đi xa hơn.

→ Nếu sử dụng **BFS** cho 1 đồ thị liên thông thì toàn bộ các đỉnh đều sẽ được thăm.

→ Độ phức tạp của thuật toán BFS là $\mathbf{O(V + E)}$

Mã giả

```cpp
const ll N = 1e6 + 5;
bool vis[N];
vector<int> adj[N];
void bfs(int s) {
    queue<int> q;

    q.push(s);
    vis[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        // do something
        for (int v : adj[u]) {
            if (!vis[v]) {
                vis[v] = true;
                q.push(v);
            }
        }
    }
}
```

c++

```python
N = 10**6 + 5
vis = [False] * N
adj = [[] for _ in range(N)]
def bfs(s):
    q = deque()
    q.append(s)
    vis[s] = True

    while q:
        u = q.popleft()

        # do something

        for v in adj[u]:
            if not vis[v]:
                vis[v] = True
                q.append(v)
```

python

→ Phần **do something** bạn có thể tùy chỉnh theo bài toán

⟶ Vậy BFS là thuật toán duyệt đồ thị theo chiều rộng sử dụng queue

7. Tổng kết

Tổng kết

Những nội dung chính

→ Hiểu được **khái niệm đồ thị** và các thành phần: đỉnh, cạnh

Tổng kết

Những nội dung chính

→ Hiểu được **khái niệm đồ thị** và các thành phần: đỉnh, cạnh

→ Biết các **dạng đồ thị**: vô hướng, có hướng, có trọng số,...

Tổng kết

Những nội dung chính

→ Hiểu được **khái niệm đồ thị** và các thành phần: đỉnh, cạnh

→ Biết các **dạng đồ thị**: vô hướng, có hướng, có trọng số,...

→ Nắm được 3 cách biểu diễn:

- Ma trận kề
- Danh sách kề
- Danh sách cạnh

Tổng kết

Những nội dung chính

→ Hiểu được **khái niệm đồ thị** và các thành phần: đỉnh, cạnh

→ Biết các **dạng đồ thị**: vô hướng, có hướng, có trọng số,...

→ Nắm được 3 cách biểu diễn:

- Ma trận kề
- Danh sách kề
- Danh sách cạnh

→ Hiểu và áp dụng 2 thuật toán:

- **DFS** – duyệt theo chiều sâu
- **BFS** – duyệt theo chiều rộng

Kết luận

Đồ thị là một công cụ mạnh mẽ giúp mô hình hóa và giải quyết nhiều bài toán thực tế trong lập trình.

Bài tập luyện tập

Các nền tảng luyện tập

- **VNOI (VOJ)**: `https://oj.vnoi.info/`
- **Codeforces**: `https://codeforces.com/`
- **AtCoder**: `https://atcoder.jp/`
- **LeetCode**: `https://leetcode.com/`

Bài tập luyện tập

Các nền tảng luyện tập

- **VNOI (VOJ)**: `https://oj.vnoi.info/`
- **Codeforces**: `https://codeforces.com/`
- **AtCoder**: `https://atcoder.jp/`
- **LeetCode**: `https://leetcode.com/`

Gợi ý bài tập BFS/DFS cơ bản

- DFS/BFS trên đồ thị liên thông
- Đếm số thành phần liên thông
- Tìm đường đi ngắn nhất (BFS)
- Kiểm tra chu trình trong đồ thị

Cảm ơn thầy cô và các bạn đã theo dõi!

Xin chân thành cảm ơn